TEMCO LEARN

T3000 API & Protocol

WebSocket API · built Nov 13, 2026

WebSocket API

Real-time bi-directional communication for live data updates.

Connection

JAVASCRIPT

```javascript
const ws = new WebSocket('ws://localhost:9103/ws');
```

Messages

Subscribe to Updates

JSON

```json
{
  "type": "subscribe",
  "serial": 237219,
  "points": ["IN1", "IN2"]
}
```

Receive Data

JSON

```json
{
  "type": "data",
  "point": "IN1",
  "value": 72.5,
  "timestamp": 1704582000
}
```

Best Practices

- Reconnect on disconnect

- Limit subscriptions
- Handle backpressure

WebSocket API

Protocol Specification

Connection Endpoint

```text
ws://localhost:9103/ws
wss://localhost:9103/ws  (TLS)
```

Message Format

All messages are JSON-encoded:

```typescript
interface WebSocketMessage {
  type: string;
  [key: string]: any;
}
```

Client-to-Server Messages

Subscribe

TYPESCRIPT

```typescript
interface SubscribeMessage {
  type: 'subscribe';
  deviceId?: number;
  serial?: number;
  points: string[];
  interval?: number;       // Update interval (ms)
  changeOfValue?: number;  // COV threshold
}

// Example
ws.send(JSON.stringify({
  type: 'subscribe',
  deviceId: 389001,
  points: ['IN1', 'IN2', 'OUT1'],
  changeOfValue: 0.5  // Only send if value changes by 0.5+
}));
```

Unsubscribe

TYPESCRIPT

```typescript
interface UnsubscribeMessage {
  type: 'unsubscribe';
  deviceId?: number;
  points?: string[];  // If omitted, unsubscribe from all
}
```

Write Point

```typescript
interface WritePointMessage {
  type: 'write';
  deviceId: number;
  point: string;
  value: number;
  priority?: number;  // BACnet priority 1-16
}
```

Server-to-Client Messages

Point Update

```typescript
interface PointUpdateMessage {
  type: 'point-update';
  deviceId: number;
  point: string;
  value: number;
  timestamp: number;
  quality: 'good' | 'uncertain' | 'bad';
  cov?: boolean;  // True if triggered by COV
}
```

Alarm Notification

```typescript
interface AlarmMessage {
  type: 'alarm';
  alarmId: string;
  deviceId: number;
  point: string;
  alarmType: 'high-limit' | 'low-limit' | 'fault';
  priority: 'high' | 'medium' | 'low';
  message: string;
  value: number;
  timestamp: number;
}
```

Device Status

TYPESCRIPT

```typescript
interface DeviceStatusMessage {
  type: 'device-status';
  deviceId: number;
  status: 'online' | 'offline' | 'error';
  lastSeen: number;
}
```

Implementation

Robust WebSocket Client

TYPESCRIPT

```typescript
class T3000WebSocketClient {
  private ws: WebSocket | null = null;
  private url: string;
  private reconnectDelay = 1000;
  private maxReconnectDelay = 30000;
  private subscriptions = new Set<string>();

  constructor(url: string) {
    this.url = url;
    this.connect();
  }

  private connect() {
    this.ws = new WebSocket(this.url);

    this.ws.onopen = () => {
      console.log('WebSocket connected');
      this.reconnectDelay = 1000;
      this.resubscribe();
    };

    this.ws.onmessage = (event) => {
      const message = JSON.parse(event.data);
      this.handleMessage(message);
    };

    this.ws.onerror = (error) => {
      console.error('WebSocket error:', error);
    };

    this.ws.onclose = () => {
      console.log('WebSocket closed, reconnecting...');
      setTimeout(() => this.connect(), this.reconnectDelay);
      this.reconnectDelay = Math.min(
        this.reconnectDelay * 2,
        this.maxReconnectDelay
```

```
      );
    };
  }

  subscribe(points: string[], deviceId: number) {
    const message = {
      type: 'subscribe',
      deviceId,
      points
    };

    // Store subscription for reconnect
    this.subscriptions.add(JSON.stringify(message));

    this.send(message);
  }

  private resubscribe() {
    for (const sub of this.subscriptions) {
      this.send(JSON.parse(sub));
    }
  }

  private send(message: any) {
    if (this.ws?.readyState === WebSocket.OPEN) {
      this.ws.send(JSON.stringify(message));
    }
  }

  on(event: string, callback: (data: any) => void) {
    // Event emitter implementation
  }
}
```

Server-Side Implementation

TYPESCRIPT

```typescript
import WebSocket from 'ws';

class WebSocketServer {
  private wss: WebSocket.Server;
  private clients = new Map<WebSocket, ClientState>();

  constructor(port: number) {
    this.wss = new WebSocket.Server({ port });

    this.wss.on('connection', (ws) => {
      const clientState: ClientState = {
        subscriptions: new Map(),
        lastHeartbeat: Date.now()
      };

      this.clients.set(ws, clientState);

      ws.on('message', (data) => {
        const message = JSON.parse(data.toString());
        this.handleMessage(ws, message);
      });

      ws.on('close', () => {
        this.clients.delete(ws);
      });
    });

    // Heartbeat to detect dead connections
    setInterval(() => this.checkHeartbeats(), 30000);
  }

  private handleMessage(ws: WebSocket, message: any) {
    switch (message.type) {
      case 'subscribe':
        this.handleSubscribe(ws, message);
        break;
      case 'unsubscribe':
        this.handleUnsubscribe(ws, message);
```

```
        break;
      case 'write':
        this.handleWrite(ws, message);
        break;
    }
  }

  broadcastPointUpdate(deviceId: number, point: string, value: number) {
    for (const [ws, state] of this.clients.entries()) {
      const subs = state.subscriptions.get(deviceId);

      if (subs?.has(point)) {
        ws.send(JSON.stringify({
          type: 'point-update',
          deviceId,
          point,
          value,
          timestamp: Date.now()
        }));
      }
    }
  }
}
```

Next Steps

- REST API
- Events
- Modbus Protocol